Technical Report: CosmicLens API

Student Name: Liu Siyuan

Student ID: 2022115992

Module: XJCO3011 - Web Services and Web Data

Assignment: Coursework 1 - Individual Web Services API Development Project

Date: April 2026

---

1. Executive Summary

This report documents the design, implementation, and deployment of CosmicLens API, a RESTful web service providing programmatic access to NASA's iconic Astronomy Picture of the Day (APOD) dataset. The API enables users to explore, search, and analyze over 30 years of astronomical imagery and scientific explanations through a well-documented, RESTful interface.

Key Achievements

- **Full CRUD Operations:** Complete Create, Read, Update, and Delete functionality for astronomy pictures and user collections
- **Rich Analytics:** Comprehensive statistics and insights including media distribution, temporal trends, and keyword analysis
- **Modern Architecture:** Built with FastAPI, PostgreSQL, and industry best practices
- **Extensive Documentation:** Interactive Swagger UI, ReDoc, and comprehensive API documentation

---

2. Technology Stack Justification

2.1 Framework: FastAPI (Python)

Choice: FastAPI over Django, Flask, or Node.js Express

Rationale:

- **Performance:** FastAPI is one of the fastest Python frameworks, comparable to Node.js and Go
- **Automatic Documentation:** Built-in Swagger UI and ReDoc generation eliminates manual documentation effort
- **Type Safety:** Full integration with Pydantic for request/response validation
- **Async Support:** Native asynchronous programming for improved concurrency
- **Modern Standards:** Based on OpenAPI (Swagger) standards

Comparison:

```
┌──────────────┬──────────────────────────────────────────────┐
│ Framework    │ Key Features                                 │
├──────────────┼──────────────────────────────────────────────┤
│ FastAPI      │ Fast, auto-docs, type hints ✓                │
│              │ Cons: Smaller ecosystem than Django          │
├──────────────┼──────────────────────────────────────────────┤
│ Django       │ Full-featured, ORM, admin panel ✓            │
│              │ Cons: Overkill for API-only, slower          │
├──────────────┼──────────────────────────────────────────────┤
│ Flask        │ Lightweight, flexible ✓                      │
│              │ Cons: Manual documentation, no type safety   │
├──────────────┼──────────────────────────────────────────────┤
│ Express      │ Popular, npm ecosystem ✓                     │
│              │ Cons: Callback hell, less type safety        │
└──────────────┴──────────────────────────────────────────────┘
```

2.2 Database: PostgreSQL

Choice: PostgreSQL over SQLite, MySQL, or MongoDB

Rationale:

- **Relational Integrity:** Complex relationships between pictures, collections, and keywords
- **Advanced Queries:** Window functions, full-text search, and aggregation capabilities
- **ACID Compliance:** Ensures data integrity for analytics queries
- **Industry Standard:** Widely used, well-documented, excellent performance
- **JSON Support:** Hybrid storage for semi-structured data when needed

Why Not NoSQL (MongoDB):

- Structured data with clear relationships
- Requires complex aggregation queries
- Assessment brief discourages NoSQL without clear justification

2.3 ORM: SQLAlchemy

Rationale:

- Mature and stable ORM with excellent documentation
- Type-safe query building
- Support for both synchronous and asynchronous operations
- Database-agnostic design

---

3. Architecture Design

3.1 Project Structure

```
cosmiclens-api/
├── app/
│   ├── main.py              # Application entry point
│   ├── config.py           # Configuration management
│   ├── database.py         # Database connection
│   ├── models/             # SQLAlchemy ORM models
│   │   ├── __init__.py
│   │   ├── picture.py
│   │   ├── collection.py
│   │   └── association.py
│   ├── schemas/            # Pydantic validation schemas
│   │   ├── __init__.py
│   │   ├── picture.py
│   │   ├── collection.py
│   │   └── analytics.py
│   ├── routers/            # API endpoint handlers
│   │   ├── __init__.py
│   │   ├── pictures.py
│   │   ├── collections.py
│   │   └── analytics.py
│   ├── services/           # Business logic
│   │   ├── __init__.py
│   │   ├── picture_service.py
│   │   ├── collection_service.py
│   │   └── analytics_service.py
│   └── scripts/            # Utility scripts
│       ├── __init__.py
│       └── init_db.py
├── docs/                  # API documentation
│   ├── API_DOCUMENTATION.md
│   └── TECHNICAL_REPORT.md
├── data/                  # Dataset storage
│   └── apod_data.csv
├── tests/                 # Unit tests
│   ├── __init__.py
│   ├── test_pictures.py
│   └── test_collections.py
├── requirements.txt
├── .env.example
└── README.md
```

3.2 Data Model

Entity Relationship Diagram (Conceptual)

```
┌──────────────────────┐        ┌──────────────────────────┐        ┌──────────────────────┐
│  AstronomyPicture    │        │   CollectionPicture      │        │     Collection       │
├──────────────────────┤        ├──────────────────────────┤        ├──────────────────────┤
│ • id (PK)            │◄───────│ • picture_id (FK)        │───────►│ • id (PK)            │
│ • date               │        │ • collection_id (FK)     │        │ • name               │
│ • title              │        │ • added_at               │        │ • description        │
```

```
│ • explanation          │          └──────────────────────────┘          │ • is_public          │
│ • media_url            │                                               │ • created_at         │
│ • hd_url               │                                               │ • updated_at         │
│ • thumbnail_url        │                                               │ • picture_count      │
│ • media_type           │                                               └──────────────────────┘
│ • copyright            │
│ • keywords             │
│ • year                 │
│ • month                │
└────────────────────────┘
```

AstronomyPicture Entity

```python
class AstronomyPicture(Base):
    __tablename__ = "astronomy_pictures"

    id: int = Column(Integer, primary_key=True, index=True)
    date: date = Column(Date, unique=True, index=True)
    title: str = Column(String(500), nullable=False)
    explanation: str = Column(Text)
    media_url: str = Column(String(1000))
    hd_url: str = Column(String(1000))
    thumbnail_url: str = Column(String(1000))
    media_type: str = Column(String(20), default="image")
    copyright: str = Column(String(500))
    keywords: str = Column(Text)
    year: int = Column(Integer, index=True)
    month: int = Column(Integer, index=True)

    # Relationships
    collections: List["Collection"] = relationship(
        "Collection",
        secondary="collection_pictures",
        back_populates="pictures"
    )
```

Collection Entity

```python
class Collection(Base):
    __tablename__ = "collections"

    id: int = Column(Integer, primary_key=True, index=True)
    name: str = Column(String(200), nullable=False)
    description: str = Column(Text)
    is_public: bool = Column(Boolean, default=True)
    created_at: datetime = Column(DateTime, default=datetime.utcnow)
    updated_at: datetime = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)
    picture_count: int = Column(Integer, default=0)

    # Relationships
```

```
    pictures: List["AstronomyPicture"] = relationship(
        "AstronomyPicture",
        secondary="collection_pictures",
        back_populates="collections"
    )
```

3.3 API Design Principles

```
┌──────────────────────────────────────────────────────────────────────┐
│                          API Design Principles                       │
├──────────────────────────────────────────────────────────────────────┤
│ ✓ RESTful Compliance     │ Proper HTTP methods (GET, POST, PUT, DELETE)  │
│ ✓ Resource-Based URLs    │ /api/v1/pictures, /api/v1/collections         │
│ ✓ Pagination            │ Consistent page and page_size parameters       │
│ ✓ Filtering             │ Support for year, month, media_type filters    │
│ ✓ Error Handling        │ Standard HTTP codes with detailed messages    │
└──────────────────────────────────────────────────────────────────────┘
```

---

4. Functionality Specification

4.1 Core Features (Minimum Requirements - Pass Grade)

Requirement	Implementation
CRUD operations	Pictures and Collections endpoints
Four API endpoints	8 Pictures + 8 Collections + 7 Analytics = 23 endpoints
JSON responses	Pydantic models for all responses
HTTP status codes	200, 201, 204, 400, 404, 422
Database integration	PostgreSQL with SQLAlchemy

4.2 Advanced Features (High Grade)

Feature	Description	Endpoint
Advanced Search	Full-text search in title, explanation, keywords	`/pictures/search`
Analytics Dashboard	Media distribution, timelines, keyword analysis	`/analytics/*`
User Collections	Personal themed collections with many-to-many relationship	`/collections/*`
Pagination	Standard pagination with metadata	All list endpoints
Input Validation	Pydantic models with detailed validation	All endpoints

4.3 API Endpoints Summary

Astronomy Pictures (8 endpoints)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                           Pictures Endpoints                             │
├──────────────────────────┬──────────────────────────────────────────────┤
│ Method │ Endpoint        │ Description                                  │
├────────┼─────────────────┼──────────────────────────────────────────────┤
│ GET    │ /pictures       │ List with pagination and filters             │
│ GET    │ /pictures/{id}      │ Get by ID                                │
│ GET    │ /pictures/date/{date} │ Get by date                            │
│ GET    │ /pictures/search     │ Full-text search                        │
│ POST   │ /pictures       │ Create new                                   │
│ PUT    │ /pictures/{id}        │ Update                                 │
│ DELETE │ /pictures/{id}        │ Delete                                 │
│ GET    │ /pictures/random      │ Random picture                         │
└────────┴─────────────────┴──────────────────────────────────────────────┘
```

Collections (8 endpoints)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Collections Endpoints                           │
├──────────────────────────┬──────────────────────────────────────────────┤
│ Method │ Endpoint        │ Description                                  │
├────────┼─────────────────┼──────────────────────────────────────────────┤
│ GET    │ /collections│ List collections                                │
│ GET    │ /collections/{id}     │ Get with pictures                      │
│ POST   │ /collections│ Create                                          │
│ PUT    │ /collections/{id}     │ Update                                 │
│ DELETE │ /collections/{id}     │ Delete                                 │
│ POST   │ /collections/{id}/pictures │ Add picture                       │
│ DELETE │ /collections/{id}/pictures/{pic_id} │ Remove picture          │
│ GET    │ /collections/{id}/pictures │ List collection pictures         │
└────────┴─────────────────┴──────────────────────────────────────────────┘
```

Analytics (7 endpoints)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Analytics Endpoints                             │
├──────────────────────────────────┬───────────────────────────────────────┤
│ Endpoint                         │ Description                           │
├──────────────────────────────────┼───────────────────────────────────────┤
│ GET /analytics/overview          │ Dataset statistics                    │
│ GET /analytics/media-types       │ Media distribution                    │
│ GET /analytics/timeline          │ Temporal trends                       │
│ GET /analytics/keywords          │ Top keywords                          │
│ GET /analytics/copyrights        │ Copyright distribution                │
│ GET /analytics/monthly-stats     │ Monthly analysis                      │
│ GET /analytics/year-summary/{year} │ Year statistics                     │
└──────────────────────────────────┴───────────────────────────────────────┘
```

---

5. Dataset Information

5.1 Source

Attribute	Value
Dataset	NASA Astronomy Picture of the Day (1995-2026)
Kaggle	https://www.kaggle.com/datasets/ahsanneural/nasa-astronomy-picture-of-the-day-1995-2026
Original Source	NASA APOD (https://apod.nasa.gov/apod/)
Records	11,186 images
Time Span	June 16, 1995 - Present

5.2 Data Schema

```
┌──────────────────┬──────────────────────────────────────────────────────┐
│ Database Schema: astronomy_pictures                                     │
├──────────────────┬──────────────────┬──────────────────────────────────┤
│ Field            │ Type             │ Description                      │
├──────────────────┼──────────────────┼──────────────────────────────────┤
│ id               │ SERIAL (PK)      │ Primary key                      │
│ date             │ DATE             │ APOD date (unique, indexed)      │
│ title            │ VARCHAR(500)     │ Title of the astronomy picture   │
│ explanation      │ TEXT             │ Scientific explanation (200-500 words) │
│ media_url        │ VARCHAR(1000)    │ Media URL                        │
│ hd_url           │ VARCHAR(1000)    │ HD version URL                   │
│ thumbnail_url    │ VARCHAR(1000)    │ Thumbnail URL                    │
│ media_type       │ VARCHAR(20)      │ image, video, other              │
│ copyright        │ VARCHAR(500)     │ Copyright holder                 │
│ keywords         │ TEXT             │ Comma-separated keywords         │
```

```
│ year            │ INTEGER         │ Year (indexed)                          │
│ month           │ INTEGER         │ Month (indexed)                          │
│ created_at      │ TIMESTAMP       │ Record creation time                      │
└─────────────────┴─────────────────┴──────────────────────────────────────────┘
```

5.3 Data Quality

Metric	Percentage
Images	96.3%
Videos	3.4%
Other	0.3%

Quality Indicators:

- Most records include HD URLs
- Extensive scientific explanations
- Keyword tagging for categorization

---

6. Testing Approach

6.1 Unit Testing

```python
# Example: Model validation with Pydantic
class TestPictureSchema:
    def test_valid_picture(self):
        picture = PictureCreate(
            date="2024-01-01",
            title="Test Picture",
            explanation="A test picture",
            media_url="https://example.com/image.jpg"
        )
        assert picture.title == "Test Picture"

    def test_invalid_date(self):
        with pytest.raises(ValidationError):
            PictureCreate(date="invalid-date", ...)
```

6.2 Manual Testing

- API endpoint testing via Swagger UI
- Database integration verification
- Error handling validation

6.3 Testing Results

```
┌──────────────────────────────────────────────────────────┐
│                    Testing Results                       │
├──────────────────────────────────────────┬───────────────┤
│ Test Category                            │ Status        │
├──────────────────────────────────────────┼───────────────┤
│ CRUD Operations                          │ ✅ PASS       │
│ Pagination                               │ ✅ PASS       │
│ Search Functionality                     │ ✅ PASS       │
│ Error Handling                           │ ✅ PASS       │
│ Database Integration                     │ ✅ PASS       │
└──────────────────────────────────────────┴───────────────┘
```

---

7. Challenges and Lessons Learned

7.1 Challenges Encountered

Challenge	Impact
Database Initialization	Setting up PostgreSQL and configuring the connection
Data Import	Processing 11,000+ records with various date formats
Media Type Detection	Automatically classifying media from URLs
Relationship Management	Implementing many-to-many relationships correctly

7.2 Solutions Implemented

```python
# Database Setup Script
python -m app.scripts.init_db

# Robust CSV Parser for handling multiple date formats
def parse_date(date_str):
    formats = ["%Y-%m-%d", "%m/%d/%Y", "%d-%b-%Y", "%B %d, %Y"]
    for fmt in formats:
        try:
            return datetime.strptime(date_str, fmt).date()
        except ValueError:
            continue
    return None

# URL-Based Classification for media type
def detect_media_type(url):
    video_exts = ['.mp4', '.youtube', '.vimeo']
    image_exts = ['.jpg', '.jpeg', '.png', '.gif', '.webp']
    # ...
```

7.3 Lessons Learned

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Key Lessons Learned                             │
├──────────────────────────────────────────────────────────────────────────┤
│ 🔷 API Design: Planning endpoints before implementation saves time       │
│ 🔷 Schema Validation: Pydantic catches errors early                      │
│ 🔷 Database Indexing: Essential for query performance at scale           │
│ 🔷 Documentation: Auto-generated docs are valuable for debugging         │
└──────────────────────────────────────────────────────────────────────────┘
```

---

8. Limitations and Future Work

8.1 Current Limitations

Limitation	Description
No Authentication	API is currently open to all users
No Rate Limiting	Potential for abuse
Single Database	No caching layer
Limited Search	No fuzzy matching or advanced NLP

8.2 Future Improvements

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        Future Enhancement Roadmap                        │
├──────────────────────────────┬──────────────────────────────────────────┤
│ Feature                      │ Technology                               │
├──────────────────────────────┼──────────────────────────────────────────┤
│ User Authentication          │ JWT-based authentication                 │
│ Rate Limiting                │ Per-user quotas (Redis)                  │
│ Caching                      │ Redis cache for frequently accessed data │
│ Enhanced Search              │ Elasticsearch or full-text search        │
│ API Versioning               │ Gradual deprecation support              │
│ Real-Time Updates            │ WebSocket support for live updates       │
│ Machine Learning             │ Recommendation engine for similar pictures │
└──────────────────────────────┴──────────────────────────────────────────┘
```

---

9. Deployment

9.1 Local Deployment

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Configure environment
cp .env.example .env
# Then edit .env with database credentials

# 3. Initialize database
python -m app.scripts.init_db

# 4. Run server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

9.2 External Hosting Options

```
┌──────────────────────┬──────────────────────────────────────────────────┐
│ Platform             │ Features                                         │
├──────────────────────┼──────────────────────────────────────────────────┤
│ PythonAnywhere       │ Free tier available, easy Python hosting         │
│ Railway              │ Easy PostgreSQL integration, modern deployment   │
│ Render               │ Good free tier, automatic HTTPS                  │
│ Heroku               │ Well-documented Python support, mature platform  │
│ Fly.io               │ Edge deployment, Docker support                  │
└──────────────────────┴──────────────────────────────────────────────────┘
```

---

10. Conclusion

CosmicLens API successfully implements a comprehensive RESTful web service for NASA's Astronomy Picture of the Day dataset. The project demonstrates:

```
┌──────────────────────────────────────────────────────────────────────┐
│                         Project Achievements                         │
├──────────────────────────────────────────────────────────────────────┤
│ ✅ Proficiency in API design and RESTful principles                  │
│ ✅ Understanding of database design with SQL                         │
│ ✅ Modern Python web development practices                           │
│ ✅ Comprehensive documentation and testing                           │
│ ✅ Implementation exceeds minimum requirements                       │
└──────────────────────────────────────────────────────────────────────┘
```

The implementation provides a solid foundation for future enhancements and showcases best practices in modern web API development.

References

1. **FastAPI Documentation.** (2024). https://fastapi.tiangolo.com/

2. **SQLAlchemy Documentation.** (2024). https://docs.sqlalchemy.org/
3. **NASA APOD Dataset.** Kaggle. https://www.kaggle.com/datasets/ahsanneural/nasa-astronomy-picture-of-the-day-1995-2026
4. **NASA APOD Official Site.** https://apod.nasa.gov/apod/
5. **REST API Design Rulebook.** Mark Masse. O'Reilly Media

Appendix A: GenAI Declaration

Tools Used

Tool	Purpose	Usage Level
ChatGPT (Cursor AI)	Code generation and debugging	High
GitHub Copilot	Code completion	Medium
Claude AI	Documentation review	Medium

Specific Applications

- **Project Structure:** AI-assisted design of project architecture
- **Code Generation:** Boilerplate code for routers and schemas
- **Debugging:** Identifying and fixing validation errors
- **Documentation:** Generating API documentation examples
- **Testing:** Creating test cases and edge cases

Ethical Considerations

- All generated code was reviewed and modified as needed
- Original NASA dataset is properly attributed
- No confidential data was processed

Appendix B: GitHub Repository

Repository: https://github.com/lsyuan777777/Web_coursework_

Repository Contents

```
📁 Web_coursework_/
├── 📄 README.md
├── 📄 requirements.txt
├── 📁 cosmiclens-api/
│   ├── 📁 app/              # Source code with complete commit history
│   ├── 📁 docs/             # API documentation (PDF)
│   ├── 📁 data/             # Sample data files
│   └── 📁 tests/            # Unit tests
└── 📄 .gitignore
```

Report generated: April 2026

Word count: Approximately 2,500 words